COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	9/16/26	**DATE SUBMITTED:**	9/16/26
NAME:	Rayven Ferry Licas	**INSTRUCTOR:**	ENGR. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Submit your basic implementation of a Binary Tree through this activity.

Your submission must contain the following:

1. Code Snippet

- Submit the complete code for your `BinaryTree` class and its `main.cpp` file.
- The code must demonstrate the creation and basic manipulation of a Binary Tree.

2. Documentation

- Submit a document explaining the methods/functions of the `BinaryTree` class you created.
- Use your Activity Template 2026

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```
main(){

    BinaryTree<char>* root = new BinaryTree<char>('A');
    root->insertChildLeft('B');
    root-> insertChildRight('C');

    root->getLeftChild()->insertChildLeft('D');
    root->getLeftChild() -> insertChildRight('E');
    root -> getRightChild()->insertChildRight('F');
```

```
    std::cout<<"The root: " << root->getRootVal() << std::endl;
    std::cout<<"The left child of root A is: " <<
root->getLeftChild()->getRootVal() << std::endl;
    std::cout<<"The left child of node B is:: " <<
root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout<<"The right child of node B is: " <<
root->getLeftChild()->getRightChild()->getRootVal() << std::endl;
    std::cout<<"The right child of root A is: " <<
root->getRightChild()->getRootVal() << std::endl;
    std::cout<<"The right child of node C is:: " <<
root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    root -> getRightChild()->setRootVal('V');
    std::cout<<"The NEW right child of root A is: " <<
root->getRightChild()->getRootVal() << std::endl;




    return 0;
}
```

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        // pointers for left and right child
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //constructor
        BinaryTree(T rootObj);
```

```
        // insert a child on the left node
        void insertChildLeft(T newNode);
        void insertChildRight(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        void setRootVal(T newNode);
};

//constructor:
template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}
// insert child:
template<typename T>
void BinaryTree<T>::insertChildLeft(T newNode){
    leftChild = new BinaryTree<T>(newNode);
}

template<typename T>
void BinaryTree<T>::insertChildRight(T newNode){
    rightChild = new BinaryTree<T>(newNode);
}

template<typename T>
BinaryTree<T> *BinaryTree<T>::getLeftChild(){
    return leftChild;
}
```

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
```

```
class BinaryTree{
    private:
        T key;
        // pointers for left and right child
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //constructor
        BinaryTree(T rootObj);
        // insert a child on the left node
        void insertChildLeft(T newNode);
        void insertChildRight(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        void setRootVal(T newNode);
};

//constructor:
template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}
// insert child:
template<typename T>
void BinaryTree<T>::insertChildLeft(T newNode){
    leftChild = new BinaryTree<T>(newNode);
}

template<typename T>
void BinaryTree<T>::insertChildRight(T newNode){
    rightChild = new BinaryTree<T>(newNode);
}
```

```cpp
template<typename T>
BinaryTree<T> *BinaryTree<T>::getLeftChild(){
    return leftChild;
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.